RAMAIAH
Institute of Technology

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Course Code: CYL75
GenAI Laboratory

Project Title: *"PhishNet" – An AI-Powered Social Engineering Simulation Game*

Literature Survey and Design/Methodology Report

Team

Name	USN
Bhaskar Datta	1MS22CY014
Dhanush J	1MS22CY021
Dushyanth S	1MS22CY025

M.S. RAMAIAH INSTITUTE OF TECHNOLOGY
(Autonomous Institute, Affiliated to VTU)

1. Literature Survey

Paper 1 — Jain, A. K. et al. (2020). *Phishing Detection Using Machine Learning*

Contributions & Techniques: Presents a feature-engineered approach to phishing email detection using Random Forest, SVM and feedforward neural networks. Extracted 47 features (header, URL, linguistic, sender reputation) and applied TF-IDF for text.
Outcomes: Random Forest gave best results with **94.2% accuracy** and low false positive rate (~2.1%). Strength: interpretability via feature importance. Limitation: relies on handcrafted features and struggles with novel linguistic obfuscations.

Paper 2 — Chen, M. et al. (2021). *Deep Learning for Phishing Email Detection*

Contributions & Techniques: Develops a hierarchical deep model (character → word → sentence) combining CNN and LSTM, uses Word2Vec embeddings and attention to capture obfuscation and context.
Outcomes: Achieved **~96.8% accuracy**, superior to classical ML. Strength: captures sequential patterns and hidden syntax used in spear-phishing. Limitation: higher compute cost and data requirement.

Paper 3 — Kumar, R. & Patel, S. (2019). *Social Engineering Attack Simulation Framework*

Contributions & Techniques: Rule-based scenario generator and training framework with 200+ templates across multiple attack types (phishing, pretexting, baiting). Tracks user responses and decision times.
Outcomes: User studies showed **~35% improvement** in recognition. Strength: practical and easy to deploy. Limitation: deterministic templates (predictable) and no real-time adaptivity.

Paper 4 — Zhang, L. et al. (2022). *RAG-Based Cybersecurity Assistant for Threat Intelligence*

Contributions & Techniques: Retrieval-Augmented Generation combining a vector DB (Chroma-style) + LLM (GPT-3.5) for contextual threat answers; uses cross-encoder re-ranking.
Outcomes: ~89% relevance accuracy, <2s average response. Strength: grounded, citeable answers; improves analyst understanding. Limitation: requires curated KB and API dependency.

Paper 5 — Williams, A. & Thompson, B. (2020). *Gamification in Cybersecurity Education*

Contributions & Techniques: Studies gamified learning (points, badges, leaderboards) across 15 modules for undergraduates. Measures engagement and retention across 12 weeks.
Outcomes: +28% engagement, +22% retention vs lecturing. Strength: clear engagement uplift. Limitation: extrinsic rewards show diminishing returns after ~8 weeks.

Paper 6 — Lee, K. et al. (2023). *LLM-Powered Conversational Agents for Security Training*

Contributions & Techniques: Uses GPT-4 style agents for multi-turn social engineering simulation with persistent persona and context management. Evaluates realism and engagement.
Outcomes: Users reported **~45% higher realism** and **~38% higher engagement** than scripted scenarios. Strength: highly varied, natural dialogues. Limitation: operational costs and latency (API cost/response time).

Paper 7 — Rodriguez, S. et al. (2021). *Multi-Modal Phishing Detection Using NLP & Vision*

Contributions & Techniques: Late-fusion of BERT textual embeddings and ResNet-50 visual features; OCR for images; fusion classifier.
Outcomes: ~97.3% accuracy, outperforming text-only. Strength: detects visual spoofing (brand impersonation). Limitation: heavy model complexity and GPU needs.

Paper 8 — Anderson, P. et al. (2022). *Adaptive Learning Systems for Cybersecurity Training*

Contributions & Techniques: RL-based personalization—MAB for content selection and DQN for difficulty adaptation—using interaction signals.
Outcomes: Users achieved competency **~32% faster** and dropout reduced by 24%. Strength: personalized learning paths. Limitation: cold-start for new users and need for substantial interaction data.

Paper 9 — Martinez, J. & Brown, A. (2020). *Real-Time Phishing Detection Using Ensemble Methods*

Contributions & Techniques: Ensemble (RF, Gradient Boosting, LR, SVM, NN) with stacking meta-learner (XGBoost) and production-oriented optimizations (feature caching, parallelization).
Outcomes: 95.7% accuracy, FPR ~1.2%, ~94 ms processing per email on CPU. Strength: production viability and low FPR. Limitation: multiple model maintenance overhead.

Paper 10 — Wang, Y. et al. (2022). *Knowledge Graph-Based Threat Intelligence System*

Contributions & Techniques: Builds Neo4j knowledge graph linking attacks, vulnerabilities, actors; uses NER and GNN inference.
Outcomes: Retrieval improvement **~52%** and faster query times; discovered hidden relations. Strength: rich semantic queries; Limitations: high construction effort and need for domain expertise.

Paper 11 — Johnson, M. et al. (2021). *Behavioral Analysis for Social Engineering Detection*

Contributions & Techniques: Uses interaction telemetry (response times, mouse/keystroke dynamics) with Isolation Forest and LSTM temporal models to detect anomalous responses to social engineering.

Outcomes: Detected **~23% more** social engineering cases vs content-only; FPR ~2.8%. Strength: content-agnostic detection. Limitation: privacy concerns and user discomfort.

Paper 12 — Singh, R. & Chen, L. (2023). *Federated Learning for Collaborative Phishing Detection*

Contributions & Techniques: Federated averaging across orgs, differential privacy (ε=1.0), secure aggregation to share gradients not raw emails.
Outcomes: 93.5% accuracy (1.2% lower than centralized) with improved cross-org generalization. Strength: privacy-preserving collaboration. Limitation: communication overhead and synchronization complexity.

Paper 13 — White, T. et al. (2022). *Explainable AI for Phishing Detection*

Contributions & Techniques: Augments transformer models with attention visualization, SHAP/LIME instance explanations and multi-level explainers (quick highlights, textual rationale).
Outcomes: Maintains **~94.1% accuracy** and improves analyst trust by ~41%. Strength: transparency and actionable insight. Limitation: marginal performance drop and additional latency.

Paper 14 — Garcia, A. et al. (2021). *Mobile-Based Phishing Awareness Training Platform*

Contributions & Techniques: Micro-learning mobile app with spaced repetition, push notifications, short interactive modules and mini-games.
Outcomes: Higher module completion (3×) and +25% retention; DAU 78%→52% over 6 months. Strength: accessibility for Gen-Z. Limitation: engagement decays over long term.

Paper 15 — Kim, H. & Park, S. (2023). *Transformer-Based Email Classification for Phishing Detection*

Contributions & Techniques: Fine-tuned RoBERTa-large and BERT on 100K emails; used augmentation (back-translation, synonym replacement), mixed precision and token scheduling.
Outcomes: State-of-the-art: 98.1% accuracy, precision 97.9%, recall 98.3%; robust to adversarial perturbations. Strength: SOTA performance. Limitation: expensive training (A100, long runtimes).

Comparative Table

	Paper (Year)	Methods / Models	Dataset (type & size)	Primary Metrics (reported)	Notable Strengths	Practical Limitations
1	Jain et al. (2020)	RF, SVM, ANN; feature engineering, TF-IDF	Email corpus, 50K	Acc 94.2%, FPR 2.1%	Interpretability; feature importance	Static handcrafted features
2	Chen et al. (2021)	Hierarchical CNN + LSTM + Attention; Word2Vec	80K labeled emails	Acc 96.8%	Good sequential/ context modeling	Compute/data heavy
3	Kumar & Patel (2019)	Rule-based scenario simulation	200 templates; user study (150)	+35% awareness	Practical training deployment	Predictable templates, not adaptive
4	Zhang et al. (2022)	RAG (vec DB + GPT-3.5)	10K threat reports	Relevance ~89%, <2s latency	Contextual grounded answers & citations	KB curation + API reliance
5	Williams & Thompson (2020)	Gamification (badges, leaderboards)	500 students (12 weeks)	+28% engagement, +22% retention	Engagement boost	Diminishing extrinsic motivation
6	Lee et al. (2023)	GPT-4 conversational agents, persona management	200 sessions, 100 participants	+45% realism, +38% engagement	Realistic multi-turn attacks	API cost & latency
7	Rodriguez et al. (2021)	BERT + ResNet-50 (late fusion) + OCR	60K emails, 25K images	Acc 97.3%	Detects visual impersonation	Complex, high GPU usage
8	Anderson et al. (2022)	RL (MAB + DQN) for adaptivity	300 users (6 months)	+32% faster skill gain	Personalized progression	Cold start; needs interactions
9	Martinez & Brown (2020)	Ensemble (RF, GB, SVM, NN) + stacking	70K emails (SMOTE balanced)	Acc 95.7%, FPR 1.2%	Low FPR; real-time capable	Maintenance complexity
10	Wang et al. (2022)	Knowledge Graph + GNN	KG: 50K nodes, 200K relations	+52% retrieval efficiency	Rich semantic queries	Heavy setup & expertise
11	Johnson et al. (2021)	Behavioral telemetry + Isolation Forest + LSTM	100K interactions, 500 users	+23% detections; FPR 2.8%	Content-agnostic detection	Privacy concerns
12	Singh & Chen (2023)	Federated Learning + DP + secure agg	10 orgs (5K–50K each)	Acc 93.5% (near centralized)	Privacy-preserving collaboration	Comm & sync overhead

13	White et al. (2022)	BERT + SHAP, LIME, attention explainers	55K emails	Acc 94.1%; +41% analyst trust	Explainability; trust boost	Slight perf drop; extra latency
14	Garcia et al. (2021)	Mobile micro-learning + spaced repetition	1,000 users (6 months)	+25% retention; 78%→52% DAU	Mobile accessibility & micro-learning	Long-term engagement decay
15	Kim & Park (2023)	RoBERTa-large (fine-tuned), augmentation	100K emails	Acc 98.1%	SOTA accuracy; adversarial robustness	Very resource intensive

Research Gaps

1. Dynamic Generative Scenarios with Adaptivity

Most existing training platforms rely on rule-based, static templates, which become predictable over time.
While LLMs introduce variability and realism, very few systems successfully integrate:

- Generative diversity (varied attack styles)
- User performance–based difficulty progression
- Reinforcement-style adaptivity
  This gap limits personalized, evolving challenge paths needed for effective skill development.

2. Multi-Modal & Multi-Vector Training

Even though phishing detection research uses multi-modal signals (text, images, URLs, OCR, visual spoofing), training systems mostly remain email-only.
Missing elements include:

- SMS/smishing simulations
- Social media phishing
- Image-based and QR-code attacks
- Attachment-level deception
  This creates a mismatch between real-world attack diversity and training coverage.

3. Explainability Built Into Learning Feedback

Explainable AI (XAI) techniques such as SHAP, attention visualization, and rationale generation are used in analyst tools, but rarely integrated into user-facing training feedback.
As a result, learners are told *what* is right or wrong, but not:

- Why an attack is deceptive
- Which red flags matter most
- How verification should be performed
  Lack of pedagogical explainability reduces knowledge transfer and long-term resilience.

These gaps form the foundation of PhishNet's objectives: **an integrated, generative, adaptive, explainable, privacy-aware, multi-modal training platform**.

2. Design & Methodology

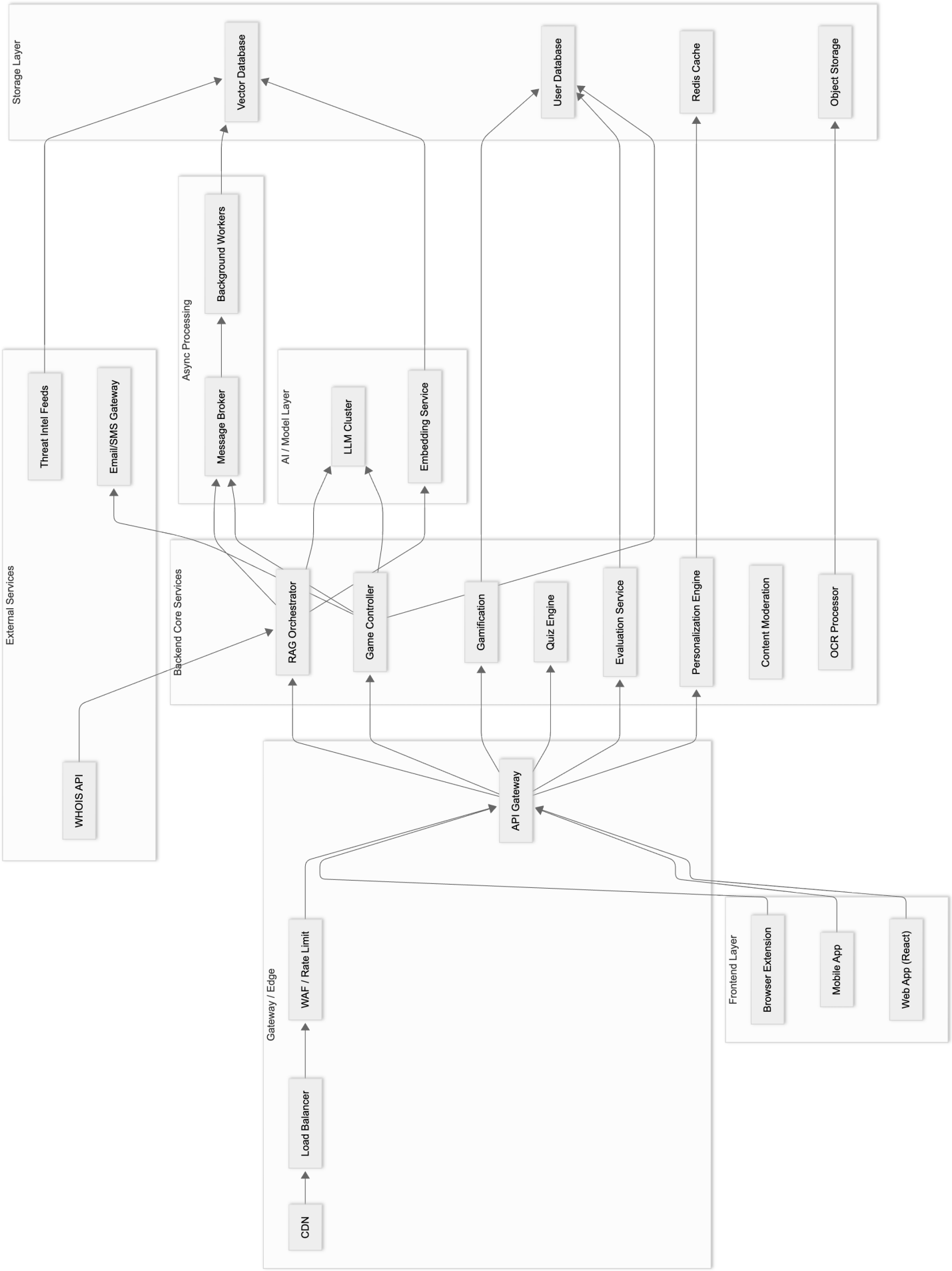
2.1 Objectives

1. To Develop a Comprehensive and Immersive Gamified Cybersecurity Education Ecosystem

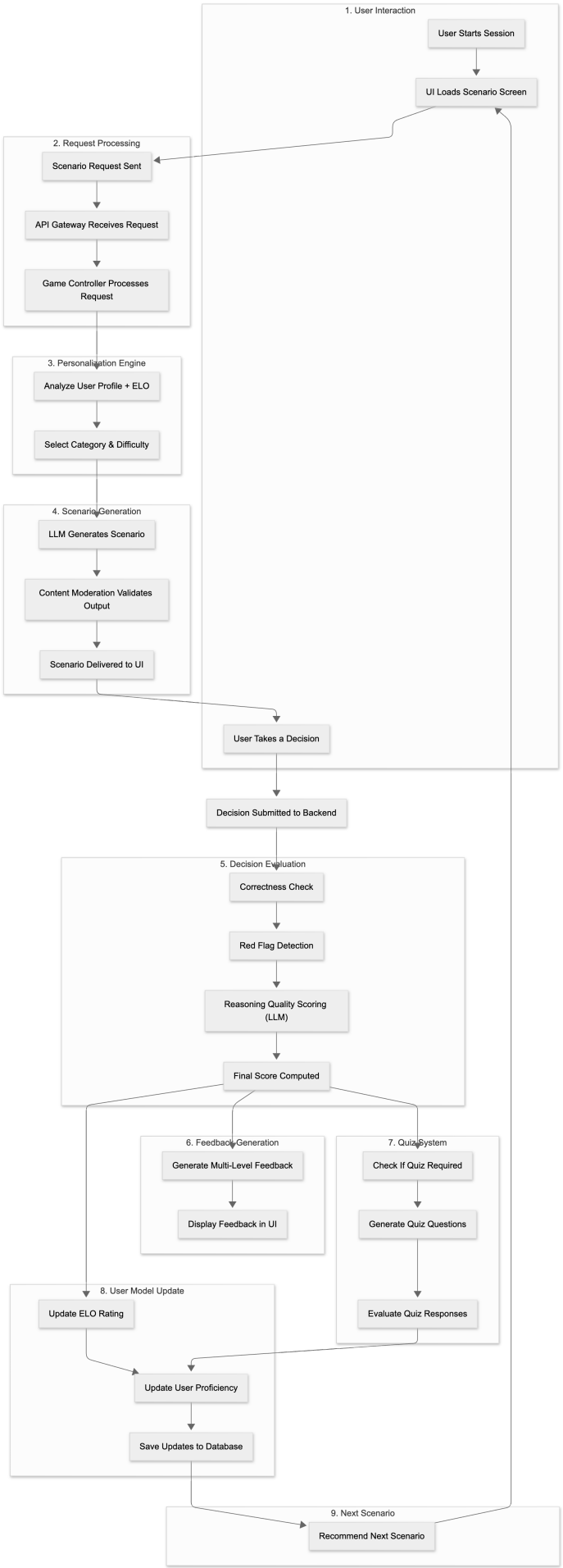
2. To Implement an Adaptively Personalized Dynamic Scenario Generation Framework Leveraging Advanced AI Mechanisms

3. To Establish an Intelligent, Multi-Layered Explanation and Assessment Mechanism for Scenario-Driven Learning

2.2 System Architecture — Block Diagram



2.3 Step-by-Step Workflow



Workflow A — Session & Scenario Generation

1. **User login / session start.** Backend fetches user profile, current ELO rating and recent history from User DB.

2. **Difficulty mapping.** ELO → difficulty level (1–5); apply streak adjustment and category weighting.

3. **Scenario parameters selected.** Category (email/sms/image), attack vector, red-flag count, personalization tokens.

4. **LLM prompt construction.** Backend builds prompt template (persona + constraints + JSON output schema) and calls LLM (Groq Llama).

5. **Scenario generation.** LLM returns scenario JSON (sender, subject/body, links, attachments, red_flags, correct_action). Backend validates and stores scenario_id in session cache.

6. **UI renders scenario.** Show formatted email/SMS, decision buttons, timer, and Digital Guardian access.

Workflow B — Digital Guardian (RAG) Assistance

1. **User query invokes DG.** Query includes last N turns and current scenario context.

2. **Query reformulation & embedding.** Use LLM or local reformulator → embed with Sentence-BERT.

3. **Semantic retrieval.** Query ChromaDB for top-k relevant KB chunks (metadata filters: category, recency).

4. **Rerank & build context.** Cross-encoder reranking (if available); pick top chunks and construct prompt with citations.

5. **LLM response.** Request concise, pedagogical answer (hint/detailed/expert) with citations. Return to UI.

Workflow C — Decision Evaluation & Feedback

1. **User submits decision (+ optional justification).** Backend records time and response.

2. **Correctness check.** Compare user choice to scenario.correct_action. Determine base result (correct/close/incorrect).

3. **Red-flag analysis.** Compare identified flags in justification vs scenario.red_flags. Compute partial credit.

4. **Scoring & ELO update.** Compute final_score using base_score, red_flag_bonus, reasoning_quality factor and time_factor. Update ELO: `Rating += K*(actual - expected)`.

5. **Feedback generation.** Compose structured feedback: decision result, missed flags, why flags matter, verification steps, suggested practice items. Use LLM for natural wording but ensure factual grounding via RAG.

6. **Persist stats & achievements.** Update User DB and show progress, unlocked badges and suggested next scenarios.

Workflow D — Knowledge Base Management (one-time + continuous)

1. **Curation.** Collect documents (CISA, blogs, advisories, academic) and metadata.

2. **Chunking & cleaning.** Split into ~512-token chunks with overlap, keep metadata.

3. **Embedding & ingest.** Generate embeddings (Sentence-BERT) and store into ChromaDB with metadata tags.

4. **Monitoring & refresh.** Schedule periodic reindexing for new threats and remove stale entries.

2.4 Module Interaction & Dataflows

Major Modules & Interfaces

- **Frontend (UI):** Game scene rendering, Digital Guardian chat, progress dashboard — communicates via REST/WebSocket with backend.

- **Game Controller (Backend):** Orchestrates scenario lifecycle, difficulty mapping, result evaluation, and achievement logic.

- **RAG Orchestrator:** Handles query reformulation, embedding requests, vector DB retrieval, context construction, and LLM prompt orchestration.

- **LLM Service:** External provider (Groq Llama 3.3) for scenario & generative outputs.

- **Embedding Service:** Sentence-BERT model (local) to vectorize queries/doc chunks.

- **Vector DB (ChromaDB):** Stores KB embeddings and supports semantic search (top-k).

- **User DB & Session Cache:** SQLite for persistent user data; Redis for session state and temporary conversation history.

Example Interactions sequence:

1. `UI → Game Controller`: start_session(user_id)

2. `Game Controller → User DB`: fetch_profile(user_id)

3. `Game Controller → LLM Service`: generate_scenario(prompt)

4. `UI ← Game Controller`: render_scenario(scenario)
5. `UI → RAG Orchestrator`: DG_query(text + context)
6. `RAG → Embedding Service`: get_embedding(query)
7. `RAG → ChromaDB`: semantic_search(embedding, filters)
8. `RAG → LLM Service`: generate_grounded_response(prompt_with_context)
9. `UI ← RAG`: show_response(answer, sources)
10. `UI → Game Controller`: submit_decision(choice, justification)
11. `Game Controller → Evaluation Service`: evaluate(choice, scenario)
12. `Evaluation → User DB`: update_profile(stats, ELO)
13. `UI ← Game Controller`: show_feedback(score, explanations, new_rating)

2.5 Implementation & Design Choices

- **LLM selection:** Groq Llama 3.3 70B (or equivalent) chosen for high-quality scenario generation and low latency; prompt templates include explicit safety and factuality constraints.
- **RAG pipeline:** All DG responses must include source chunks (citations) to ensure factual grounding and educational integrity.
- **Personalization:** ELO + streak adjustments form fast converging difficulty mapping; long-term personalization uses category-wise proficiency stats.
- **Privacy:** By default, store only non-sensitive aggregate metrics; optional advanced telemetry is opt-in and processed on device where feasible.
- **Explainability:** Feedback contains three levels — Quick Tip, Educated Explanation, and Deep Dive — to suit novice→expert learners.
- **Scalability:** Use caching for repeated RAG queries and scenario templates; vector DB indexed for low-latency retrieval.

2.6 Success Criteria & Evaluation Plan

- **System metrics:** Scenario gen time < 3s; DG response < 2s; retrieval latency < 0.5s.
- **Learning metrics:** Pre/post accuracy improvement > 20%; retention test >70% after one week.
- **Engagement metrics:** DAU > 60% within pilot cohort; completion rate > 75% for 10+ scenarios.
- **Qualitative:** User satisfaction > 4.0/5 for scenario realism and DG helpfulness.